

Report for the Course on Cyber-Physical Systems and IoT Security

Second mid-term project



**Professional
Security
Corporation**

Members

Emanuele Artusi | 2198545

Filippo Bellon | 2199368

Reference paper

An Anomaly-Based IDS
for Detecting Attacks in
RPL-Based Internet of Things

GitHub Repository

<https://github.com/ProSecCorp/Part-2-IoT>



Contents

1	Introduction	2
2	Background	2
2.1	The RPL routing protocol	2
2.1.1	DODAG construction and topology management	2
2.1.2	RPL control messages	2
2.2	Routing attacks in RPL networks	3
2.2.1	Blackhole attack	3
2.2.2	Sinkhole attack	3
2.3	IDS in IoT	3
3	Objectives	3
4	System setup	4
4.1	Experimental environment	4
4.2	Implementation modifications	4
4.2.1	Blackhole attack	4
4.2.2	Sinkhole attack	5
4.2.3	MRHOF path-cost modification	6
4.2.4	Build and logging configuration	6
4.3	IDS implementation	6
4.3.1	Blackhole IDS	7
4.3.2	Sinkhole IDS	7
5	Experiments	7
5.1	Experimental setup	7
5.2	Experimental scenarios	7
6	Results and discussion	7
6.1	Blackhole attack	8
6.2	Sinkhole attack	9
6.3	IDS effectiveness	11
	Acronyms	14
	References	15

1 Introduction

The Internet of Things (IoT) has evolved into a widespread paradigm connecting large numbers of resource-constrained devices and enabling the development of intelligent cyber-physical systems. Many IoT deployments rely on Low-Power and Lossy Networks (LLNs), where nodes operate under strict limitations in terms of processing power, memory, energy, and communication bandwidth. Technologies such as IPv6 over Low-Power Wireless Personal Area Networks (6LoWPAN) enable Internet Protocol Version 6 (IPv6) communication in these constrained environments, while the Routing Protocol for Low-Power and Lossy Networks (RPL), standardized by the Internet Engineering Task Force (IETF), provides distributed routing and topology formation.

The characteristics of LLNs and the distributed nature of RPL introduce significant security challenges. In particular, compromised internal nodes can legitimately participate in the routing process while manipulating RPL control information, making conventional security mechanisms insufficient against internal routing attacks [1]. Among these attacks, Sinkhole and Blackhole attacks are particularly relevant. A Sinkhole node can advertise an artificially attractive routing metric to attract network traffic through itself, while a Blackhole node can exploit its position in the routing topology to selectively or completely discard forwarded packets.

This work investigates the behavior of a RPL network under Sinkhole and Blackhole attacks and evaluates their impact on network performance. In addition, a lightweight log-based Intrusion Detection System (IDS) is developed to identify routing anomalies by monitoring RPL control messages and detecting inconsistencies in node and parent ranks.

2 Background

This section introduces the concepts required to understand the experimental work.

2.1 The RPL routing protocol

The RPL is a distance-vector routing protocol designed for resource-constrained LLNs. RPL organizes nodes into a Destination-Oriented Directed Acyclic Graph (DODAG), whose topology is rooted at a node providing connectivity towards an external network.

2.1.1 DODAG construction and topology management

Each node in a DODAG is assigned a *Rank*, which represents its position relative to the root. The root has the lowest Rank, while nodes farther from the root generally have higher Rank values. Rank is used to maintain the directed acyclic structure of the DODAG and to support loop avoidance.

The selection of a preferred parent and the computation of the resulting Rank are determined by an *Objective Function (OF)*, which evaluates the routing cost associated with neighboring nodes. Two OFs commonly associated with RPL are:

- **Objective Function Zero (OF0):** primarily considers the number of hops towards the root when selecting a parent;
- **Minimum Rank with Hysteresis Objective Function (MRHOF):** considers link-quality metrics such as the Expected Transmission Count (ETX) to compute the path cost and applies hysteresis to limit unnecessary parent changes.

In the RPL implementation considered in this work, MRHOF is used for parent selection. The advertised Rank of a neighbor therefore represents an important component of the routing cost considered when evaluating potential parents.

2.1.2 RPL control messages

RPL uses Internet Control Message Protocol for IPv6 (ICMPv6) control messages to construct and maintain the DODAG:

- **DODAG Information Object (DIO):** advertises DODAG information, including the current Rank, version, and other configuration parameters. Nodes use received DIOs to discover potential parents;
- **DODAG Information Solicitation (DIS):** requests DIO messages from neighboring nodes, for example when joining or recovering from a disconnected state;

- **Destination Advertisement Object (DAO):** advertises routing information towards the root, allowing downward routes to be maintained;
- **Destination Advertisement Object Acknowledgment (DAO-ACK):** optionally acknowledges the reception of a DAO.

Among these messages, the DIO is particularly relevant to the attacks considered in this work, since it contains the Rank advertised by a node and is therefore involved in parent selection.

2.2 Routing attacks in RPL networks

The distributed nature of RPL allows a compromised internal node to participate in the routing protocol while providing malicious routing information. Two routing attacks are considered in this project: Blackhole and Sinkhole.

2.2.1 Blackhole attack

A Blackhole attack consists of a compromised routing node that discards packets that it is expected to forward instead of delivering them towards their destination. The malicious node may continue to participate normally in RPL topology formation, while disrupting data traffic that is routed through it.

The attack primarily affects packet forwarding and can therefore result in packet loss and reduced network delivery performance.

2.2.2 Sinkhole attack

A Sinkhole attack attempts to attract network traffic by making a compromised node appear to provide a highly favorable route towards the DODAG root. In RPL, this can be achieved by manipulating the Rank advertised in DIO messages.

The attacker advertises a Rank lower than its actual Rank, causing neighboring nodes to potentially consider it a more attractive parent. Once selected as a preferred parent, the malicious node can attract traffic originating from other nodes in its neighborhood.

The effectiveness of the attack depends on the routing OF, link metrics, and parent-selection mechanisms. In particular, the hysteresis mechanism of MRHOF can delay parent changes when the new route does not provide a sufficiently large improvement.

A Sinkhole attack can also be combined with a Blackhole attack, where the Sinkhole redirects traffic towards the compromised node and the Blackhole subsequently discards it.

2.3 IDS in IoT

IDS provide an additional layer of defense by monitoring network activity and identifying behavior that deviates from expected protocol operation.

IDS approaches can be deployed directly on network nodes or at a centralized monitoring point. They can also rely on different detection strategies, such as predefined protocol rules or the identification of anomalous behavior.

In this work, the IDS is based on RPL routing information and focuses on identifying inconsistencies in the observed routing hierarchy. In particular, Rank information advertised through DIO messages and parent relationships reported through DAO messages can be used to assess whether the resulting topology is consistent with the expected RPL hierarchy.

3 Objectives

The main objective of this project is to investigate the impact of routing attacks on RPL-based IoT networks and to evaluate the feasibility of detecting such attacks through routing information.

The specific objectives are:

- **Analyze RPL behavior:** study the construction and evolution of the DODAG under normal operating conditions;
- **Evaluate routing attacks:** implement and analyze Blackhole and Sinkhole attacks and observe their effects on the routing topology and packet delivery;

- **Compare network performance:** evaluate the impact of the attacks using metrics such as Packet Delivery Ratio (PDR), End-to-End Delay, and Throughput;
- **Develop a lightweight IDS:** analyze Contiki-NG execution logs to identify routing anomalies, with particular focus on inconsistencies between node and parent Rank values;
- **Assess detection capability:** evaluate whether the proposed IDS can identify the routing inconsistencies introduced by a Sinkhole attack.

4 System setup

4.1 Experimental environment

We deployed the experimental environment on a Windows 10 host using Windows Subsystem for Linux 2 (WSL2). We used an Ubuntu Linux environment to build and execute the Contiki-NG source code and its simulation tools.

We simulated the RPL network using Cooja, the network simulator included with Contiki-NG. Cooja allowed us to execute multiple Contiki-NG motes simultaneously while providing access to their network activity and execution logs.

To run the graphical Cooja interface through WSL2, we configured the display variable before launching the simulator:

```
export DISPLAY=:0
```

We then started the simulator from the Contiki-NG directory using:

```
cd tools/cooja
./gradlew run
```

We used the RPL-UDP example as the basis for our experiments. We created custom Cooja simulation configurations to define the network topology, participating motes, and attack scenarios. The simulations produced mote execution logs, which we subsequently analyzed to evaluate network performance and detect routing anomalies.

4.2 Implementation modifications

To implement the two attacks considered in our experiments, we modified selected components of the Contiki-NG networking and RPL stack. The modifications were kept localized to the routing and forwarding mechanisms, while the normal application and RPL behavior was preserved whenever possible. You can find the complete source code of the modified Contiki-NG files in the Source/Contiki patches directory of the accompanying repository.

4.2.1 Blackhole attack

We implemented the Blackhole attack at the IPv6 forwarding layer in `os/net/ipv6/uiplib.c`. The attack is enabled through a compile-time configuration and is associated with a specific simulated mote (as for the Sinkhole attack):

```
#define BLACKHOLE_ID 5
#define B_ATTACK_ACTIVE 1
```

When the malicious mote forwards a packet, we inspect the actual transport protocol. This also accounts for IPv6 packets containing a Hop-by-Hop extension header:

```
if(B_ATTACK_ACTIVE && simMoteID == BLACKHOLE_ID) {
    uint8_t real_proto = UIP_IP_BUF->proto;
    if(real_proto == UIP_PROTO_HBHO) {
        struct uip_ext_hdr *ext_hdr;
        ext_hdr = (struct uip_ext_hdr *)
            ((uint8_t *)UIP_IP_BUF + UIP_IPH_LEN);
        real_proto = ext_hdr->next;
```

```
}
if(real_proto == UIP_PROTO_UDP) {
    LOG_INFO("BLACKHOLE DROP UDP forwarding\n");
    goto drop;
}
}
```

The check was inserted in the IPv6 forwarding path, before the packet is sent to the next hop. Consequently, User Datagram Protocol (UDP) packets that reach the malicious node for forwarding are discarded, while the node can continue to participate in RPL and generate its own traffic.

We also added a dedicated `blackhole.c` application for the malicious mote. Its main purpose is to initialize the node and its participation in the routing stack:

```
PROCESS(blackhole_process, "Blackhole node");
AUTOSTART_PROCESSES(&blackhole_process);
PROCESS_THREAD(blackhole_process, ev, data)
{
    PROCESS_BEGIN();
    LOG_INFO("Blackhole started\n");
    NETSTACK_ROUTING.init();
    while(1) {
        PROCESS_WAIT_EVENT();
    }
    PROCESS_END();
}
```

The attack-specific forwarding behavior therefore remains in the IPv6 stack, rather than being implemented as an application-level packet drop.

In addition to the standard Blackhole behavior described above, we also implemented an *Aggressive Blackhole* variant in `os/net/ipv6/tcpip.c`. This variant modifies the forwarding behavior more aggressively in order to discard every packet reaching the compromised node. The implementation was retained as an additional attack configuration, but it was not evaluated in the experimental scenarios reported in this work because it is less realistic and more easily detectable than the standard Blackhole attack, since it will break the DODAG construction and prevent the malicious node from participating in the routing process.

4.2.2 Sinkhole attack

We implemented the Sinkhole attack by modifying the RPL DIO generation mechanism in `os/net/routing/rpl-lite/rpl-icmp6.c`. Normally, the advertised Rank corresponds to the node's actual Rank:

```
uint16_t advertised_rank = curr_instance.dag.rank;
```

For the malicious node, the attack is activated by replacing the advertised Rank with `ROOT_RANK`:

```
if(simMoteID == SINKHOLE_ID &&
    curr_instance.dag.rank != ROOT_RANK /*&&
    curr_instance.dag.rank != RPL_INFINITE_RANK*/) {
    advertised_rank = ROOT_RANK /*+
        curr_instance.min_hoprankinc*/;
    LOG_INFO("SINKHOLE ACTIVE fake_rank=%u "
        "real_rank=%u\n",
        advertised_rank,
        curr_instance.dag.rank);
}
```

The commented conditions represent the behavior that would be preferable for a stealthier attack. In particular, excluding `RPL_INFINITE_RANK` would prevent the attack from becoming active while the DODAG is still being constructed, when the node has not yet selected a valid parent. Similarly, adding

`curr_instance.min_hoprankinc` to the advertised Rank would avoid advertising exactly the root Rank while still providing an artificially attractive routing metric.

For the experiments, however, these conditions and the Rank increment were left commented out. This makes the Sinkhole behavior more pronounced and therefore easier to observe and analyze in the simulation logs.

The malicious node therefore advertises a Rank corresponding to the root while maintaining its actual Rank internally. This directly affects the routing information received by neighboring nodes through DIO messages.

We also modified the Metric Container of the DIO. When MRHOF uses ETX, the Sinkhole advertises a fixed ETX value corresponding to 1.0 instead of its actual path metric:

```
uint16_t fake_etx = curr_instance.mc.obj.etx;
if(S_ATTACK_ACTIVE &&
    simMoteID == SINKHOLE_ID) {
    fake_etx = 128;
    LOG_INFO("SINKHOLE: advertising fake ETX %u "
             "(real %u)\n",
             fake_etx,
             curr_instance.mc.obj.etx);
}
```

The two manipulated values, Rank and ETX, are therefore inserted directly into the RPL control information advertised by the malicious node.

4.2.3 MRHOF path-cost modification

Since the experiments use the MRHOF, we also modified its path-cost calculation in

`os/net/routing/rpl-lite/rpl-mrhof.c`.

We introduced a configurable multiplier for the link metric:

```
#define LINK_METRIC_MULTIPLIER 0
```

The original path-cost calculation was modified as follows:

```
return MIN((uint32_t)base +
           link_metric_to_rank(nbr_link_metric(nbr))
           * LINK_METRIC_MULTIPLIER,
           0xffff);
```

With the multiplier set to zero, the link-metric contribution is removed from the resulting path cost:

$$C_{\text{path}} = R_{\text{parent}} \quad (1)$$

This configuration allowed us to isolate the effect of the Rank advertised by the Sinkhole and reduce the influence of link-quality variations during the experiments. More details in Section 6.2.

4.2.4 Build and logging configuration

We modified the `examples/rpl-udp/Makefile` to include the dedicated Blackhole node and to enable the logging required during the experiments:

```
CONTIKI_PROJECT = udp-client udp-server blackhole
all: $(CONTIKI_PROJECT)
CONTIKI=../..
CFLAGS += -DLOG_CONF_LEVEL_RPL=LOG_LEVEL_DBG
CFLAGS += -DBLACKHOLE_NODE
include $(CONTIKI)/Makefile.include
```

We also enabled debug-level logging in the modified source files. This provided the RPL and forwarding information required to reconstruct the routing behavior from the simulation logs.

4.3 IDS implementation

We developed two lightweight, centralized IDSs in Python, operating on the execution logs generated by the Contiki-NG simulations. The two IDSs target the different characteristics of the attacks considered in this work.

4.3.1 Blackhole IDS

The Blackhole IDS, implemented in `Source/IDSs/IDS-blackhole.py`, monitors packet forwarding activity. It counts packets received for forwarding and packets subsequently forwarded by each node:

```
received_forward[node] += 1
forwarded[node] += 1
```

For each node, a forwarding ratio is calculated. A node is flagged when it receives more than five packets and forwards less than 60% of them. The IDS also computes PDR, end-to-end delay, and throughput and stores the results in `ids_blackhole_results.csv`.

4.3.2 Sinkhole IDS

The Sinkhole IDS, implemented in `Source/IDSs/IDS-sinkhole.py`, reconstructs the RPL topology from DIO and DAO messages by extracting node Ranks and observed parent relationships.

The main detection rule checks the consistency between a node and its parent:

```
if parent_rank >= child_rank:
    alerts.append(...)
```

Such a relationship violates the expected RPL Rank hierarchy and is reported as a potential routing anomaly. The IDS also records significant Rank changes as secondary indicators, while accounting for the normal Rank initialization that occurs during DODAG formation. The observed Rank history is exported to `ids_sinkhole_results.csv`.

5 Experiments

5.1 Experimental setup

All experiments were performed using Cooja with the same simulation configuration, with each simulation running for at least five minutes. The radio medium was configured as Uniform Distributed Grid Model (UDGM), each mote was started with a delay of 1 second, and the random seed was fixed to 123456 in order to ensure reproducibility.

The experiments were based on the RPL-UDP example included in Contiki-NG. The clients and server used in the simulations are Cooja motes implemented in `examples/rpl-udp`, with the server acting as the DODAG root and the clients participating as RPL nodes. Attack-specific motes were added when required by the scenario.

For each experiment, we collected the Cooja `.csc` configuration (in `Source/Simulations`), a screenshot of the simulated network, the Directed Acyclic Graph (DAG) reconstructed from the execution logs (in `Data/Images/`, see Figure 1 as an example), the complete mote log (in `Source/Logs`), and all the files generated by the Python IDSs (in `Source/Results`).

5.2 Experimental scenarios

The experimental campaign consists of one baseline scenario and eight attack scenarios. In Table 1, *B* denotes a Blackhole node and *S* denotes a Sinkhole (client) node. The position refers to the attacker's position within the resulting network topology. See in Figure 2c and in Figure 2f that the server node can reach only the attacker node, thus creating a communication bottleneck through the attacker.

6 Results and discussion

Note that every result presented in this section is based on the IDSs output generated by the Python scripts, which are available in `Source/Results`.

In all experiments, `IDS_sinkhole.py` reported at least one [ALERT] suspicious rank decrease for each simulated node. This behavior was also observed in the baseline scenario and is therefore not sufficient, by itself, to identify a Sinkhole attack.

The false positives are mainly caused by the DODAG formation process. Nodes initially have `RPL_INFINITE_RANK` and acquire a finite Rank when they receive the first valid DIO. The IDS currently interprets this transition

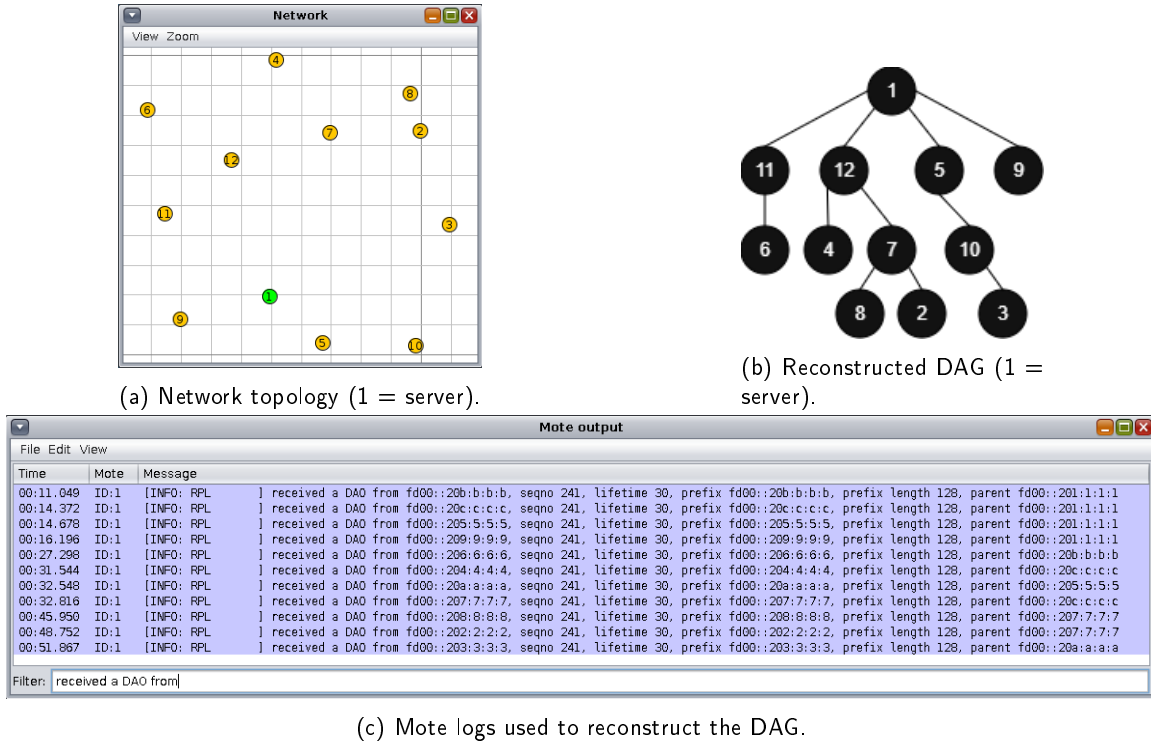


Figure 1: Baseline network.

Table 1: Experimental scenarios

Scenario	Server	Clients	Attackers	Position / Configuration
Baseline	1	11	0	–
Blackhole 1	1	10	1 B	Middle
Blackhole 2	1	10	1 B	Leaf
Blackhole 3	1	10	1 B	Bottleneck
Sinkhole 1	1	10	1 S	Middle
Sinkhole 2	1	10	1 S	Leaf
Sinkhole 3	1	10	1 S	Bottleneck
Sinkhole 4	1	10	1 S	Original MRHOF + hysteresis
Blackhole 4	1	10	1 B+S	Middle

as a suspicious Rank decrease. Therefore, Rank-decrease detection is considered too sensitive when applied during the initial DODAG construction and must be interpreted together with the Rank consistency check.

We report the results of the experimental campaign in Table 2.

6.1 Blackhole attack

An example of a packet drop caused by a Blackhole attack is shown in Figure 3.

We observed that the PDR is significantly affected by the Blackhole attack, as expected, since the attacker drops all the packets it receives for forwarding. The IDS correctly detected the Blackhole attack in scenarios 1 and 3, while it failed to detect the attack in scenario 2, where the attacker was a leaf node. In this case, the attacker did not have any children and therefore did not receive packets requiring forwarding. Consequently, the forwarding-based detection mechanism had no sufficient evidence to identify the malicious behavior. Nevertheless, the significant reduction in PDR would have provided a clear indication of the attack if this metric had also been monitored by the IDS.

The results hence show that the position of the attacker has a significant impact on the effectiveness of the attack. In scenario 1, the attacker was located in the middle of the topology, causing only part of the traffic to be dropped and resulting in a PDR of approximately 60%. In scenario 3, the attacker was positioned at a bottleneck, causing all traffic to pass through it and resulting in a PDR of 0%.

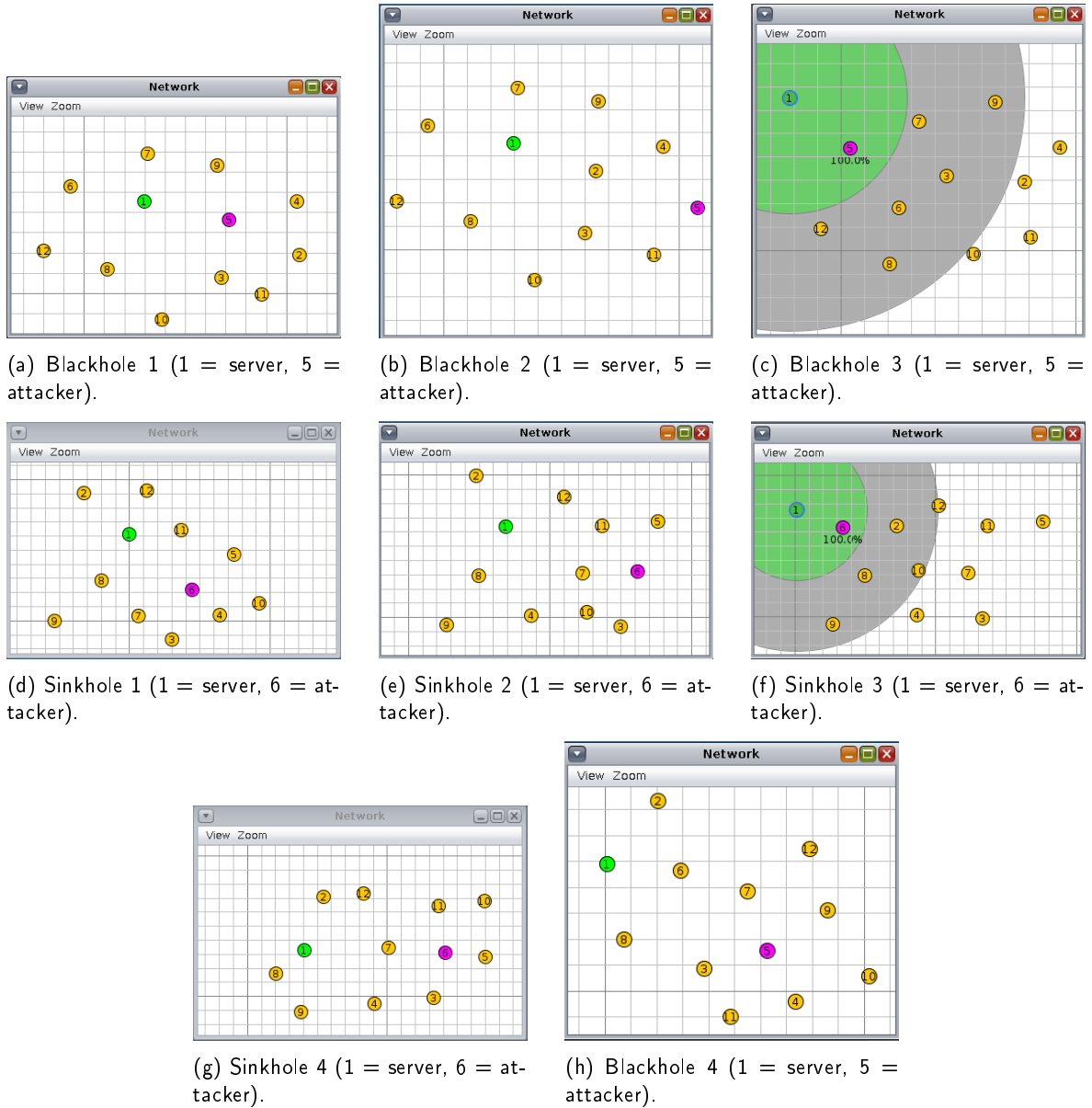


Figure 2: Network topologies for the attack scenarios.

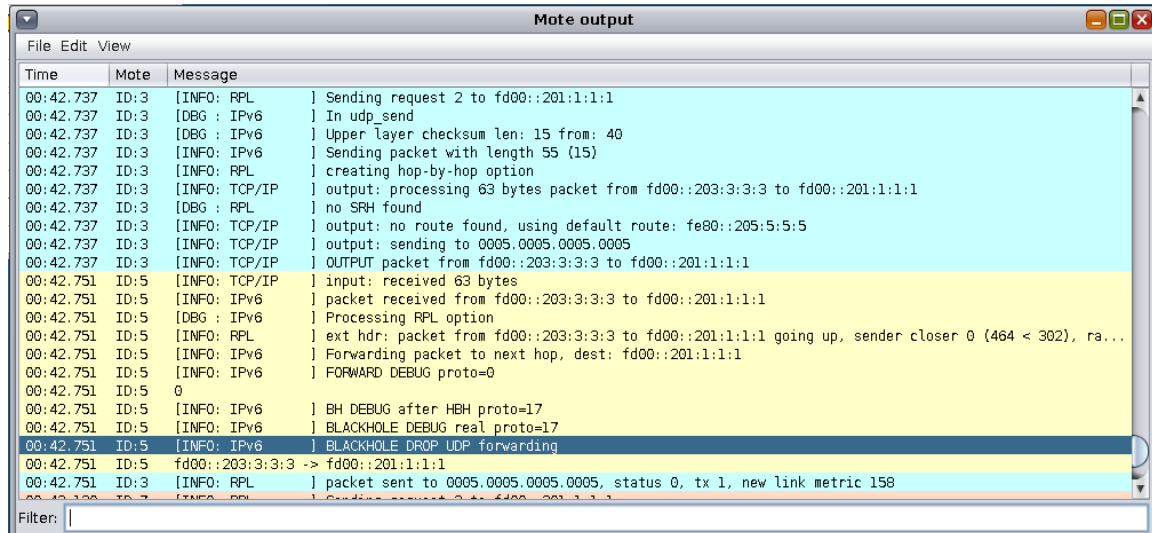
These results confirm that a Blackhole attack is strongly dependent on the attacker's position within the routing topology. A node that is not part of any forwarding path may have no observable impact on network performance.

In scenario 4, the Blackhole attack was combined with a Sinkhole attack. The Sinkhole component was correctly identified by the IDS, while the Blackhole behavior was not detected. In particular, the forwarding ratio of the suspected node remained above the detection threshold, with a value of approximately 1.09. Therefore, the forwarding-based detection rule did not classify the node as malicious, despite the presence of the Blackhole behavior. Again, this result highlights the importance of monitoring the PDR as an additional metric to detect Blackhole attacks, especially when they are combined with other attack types.

6.2 Sinkhole attack

An example of a Rank inconsistency caused by a Sinkhole attack is shown in Figure 4.

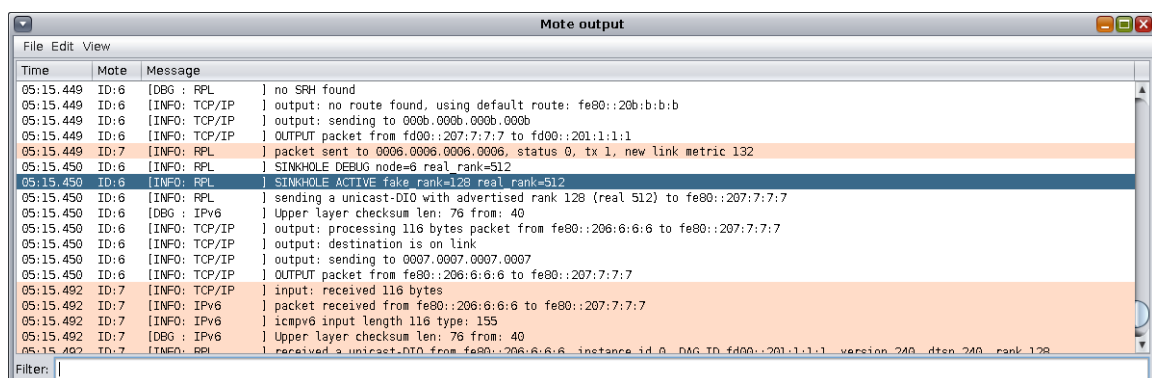
The Sinkhole scenarios produced a different behavior compared to the Blackhole experiments. As expected, the attack did not directly affect packet delivery: the PDR and throughput remained approximately unchanged across all Sinkhole scenarios. The main effect of the attack was instead observed in the routing topology, as the malicious node attempted to attract neighboring nodes by advertising an artificially favorable Rank.



```

Time      Mote    Message
00:42.737 ID:3    [INFO: RPL] Sending request 2 to fd00::201:1:1:1
00:42.737 ID:3    [DBG : IPv6] In udp_send
00:42.737 ID:3    [DBG : IPv6] Upper layer checksum len: 15 from: 40
00:42.737 ID:3    [INFO: IPv6] Sending packet with length 55 (15)
00:42.737 ID:3    [INFO: RPL] creating hop-by-hop option
00:42.737 ID:3    [INFO: TCP/IP] output: processing 63 bytes packet from fd00::203:3:3:3 to fd00::201:1:1:1
00:42.737 ID:3    [DBG : RPL] no SRH found
00:42.737 ID:3    [INFO: TCP/IP] output: no route found, using default route: fe80::205:5:5:5
00:42.737 ID:3    [INFO: TCP/IP] output: sending to 0005.0005.0005.0005
00:42.737 ID:3    [INFO: TCP/IP] OUTPUT packet from fd00::203:3:3:3 to fd00::201:1:1:1
00:42.751 ID:5    [INFO: TCP/IP] input: received 63 bytes
00:42.751 ID:5    [INFO: IPv6] packet received from fd00::203:3:3:3 to fd00::201:1:1:1
00:42.751 ID:5    [DBG : IPv6] Processing RPL option
00:42.751 ID:5    [INFO: RPL] ext hdr: packet from fd00::203:3:3:3 to fd00::201:1:1:1 going up, sender closer 0 (464 < 302), ra...
00:42.751 ID:5    [INFO: IPv6] Forwarding packet to next hop, dest: fd00::201:1:1:1
00:42.751 ID:5    [INFO: IPv6] FORWARD DEBUG proto=0
00:42.751 ID:5    0
00:42.751 ID:5    [INFO: IPv6] BH DEBUG after HBH proto=17
00:42.751 ID:5    [INFO: IPv6] BLACKHOLE DEBUG real proto=17
00:42.751 ID:5    [INFO: IPv6] BLACKHOLE DROP UDP forwarding
00:42.751 ID:5    fd00::203:3:3:3 -> fd00::201:1:1:1
00:42.751 ID:3    [INFO: RPL] packet sent to 0005.0005.0005.0005, status 0, tx 1, new link metric 158
00:42.751 ID:3    [INFO: RPL] Sending request 2 to fd00::201:1:1:1
  
```

Figure 3: Mote logs showing a packet drop caused by a Blackhole attack (scenario 1).



```

Time      Mote    Message
05:15.449 ID:6    [DBG : RPL] no SRH found
05:15.449 ID:6    [INFO: TCP/IP] output: no route found, using default route: fe80::20b:b:b:b
05:15.449 ID:6    [INFO: TCP/IP] output: sending to 000b.000b.000b.000b
05:15.449 ID:6    [INFO: TCP/IP] OUTPUT packet from fd00::207:7:7:7 to fd00::201:1:1:1
05:15.449 ID:7    [INFO: RPL] packet sent to 000b.000b.000b.000b, status 0, tx 1, new link metric 132
05:15.450 ID:6    [INFO: RPL] SINKHOLE DEBUG node=6 real rank=512
05:15.450 ID:6    [INFO: RPL] SINKHOLE ACTIVE (old rank=128, real rank=512)
05:15.450 ID:6    [INFO: RPL] sending a unicast-DIO with advertised rank 128 (real 512) to fe80::207:7:7:7
05:15.450 ID:6    [DBG : IPv6] Upper layer checksum len: 76 from: 40
05:15.450 ID:6    [INFO: TCP/IP] output: processing 116 bytes packet from fe80::206:6:6:6 to fe80::207:7:7:7
05:15.450 ID:6    [INFO: TCP/IP] output: destination is on link
05:15.450 ID:6    [INFO: TCP/IP] output: sending to 0007.0007.0007.0007
05:15.450 ID:6    [INFO: TCP/IP] OUTPUT packet from fe80::206:6:6:6 to fe80::207:7:7:7
05:15.492 ID:7    [INFO: TCP/IP] input: received 116 bytes
05:15.492 ID:7    [INFO: IPv6] packet received from fe80::206:6:6:6 to fe80::207:7:7:7
05:15.492 ID:7    [INFO: IPv6] icmpv6 input length 116 type: 155
05:15.492 ID:7    [DBG : IPv6] Upper layer checksum len: 76 from: 40
05:15.492 ID:7    [INFO: RPL] received a unicast-DIO from fe80::206:6:6:6 instance id 0 DAG-ID fd00::201:1:1:1 version 240 dten 240 rank 128
  
```

Figure 4: Mote logs showing a Rank inconsistency caused by a Sinkhole attack (scenario 2).

Table 2: Experimental results

Scenario	PDR	Throughput	Attack detected	Malicious node(s) identified
Baseline	100%	≈ 1 pkt/s	None	–
Blackhole 1	60%	≈ 0.59 pkt/s	Blackhole	Correctly
Blackhole 2	100%	≈ 0.99 pkt/s	None	–
Blackhole 3	0%	0 pkt/s	Blackhole	Correctly
Sinkhole 1	100%	≈ 1 pkt/s	Sinkhole	Correctly
Sinkhole 2	100%	≈ 1 pkt/s	Sinkhole	Correctly
Sinkhole 3	100%	≈ 1 pkt/s	Sinkhole	Correctly
Sinkhole 4	100%	≈ 1 pkt/s	Sinkhole	Correctly
Blackhole 4	49%	≈ 0.49 pkt/s	Sinkhole	Partially

The reconstructed DAGs before and after the attack are shown for each scenario in order to highlight the changes introduced in the routing topology. Together with the topology changes, the IDS was able to identify Rank inconsistencies involving the malicious node.

In the first three scenarios, the Sinkhole was placed at different positions in the topology. In the bottleneck configuration, where the malicious node was a direct child of the root, the attack was detected because the malicious node advertised `ROOT_RANK` as its Rank. This resulted in an explicit Rank inconsistency between the malicious node and its parent.

However, the effectiveness of the attack also depends on how the advertised Rank is used by the MRHOF parent-selection mechanism. In the original implementation, the path cost of a neighbor is computed from both its advertised Rank and its link metric:

$$\text{path cost} = \text{advertised Rank} + \text{ETX} \quad (2)$$

Furthermore, MRHOF applies a hysteresis mechanism that prevents a node from changing its preferred parent when the improvement in path cost is not sufficiently large. Therefore, simply advertising a moderately lower Rank does not necessarily cause neighboring nodes to select the malicious node as their parent.

For the first three Sinkhole scenarios, we therefore used an aggressive configuration in which the link-metric contribution was disabled and the malicious node advertised `ROOT_RANK`. This maximized the difference between the malicious route and the currently selected route, making the effect of the attack on the topology clearly observable.

This also explains the behavior observed in the fourth Sinkhole scenario. Here, the original MRHOF path-cost calculation and hysteresis mechanism were preserved, while the malicious node used the less aggressive Rank calculation based on `ROOT_RANK + min_hoprankinc`. When placed at the bottleneck, this configuration did not provide a sufficiently attractive alternative to the existing parent, and the resulting DAG remained substantially unchanged. Consequently, the attack had little practical effect in that position.

When the same configuration was instead used with the malicious node placed near a leaf, the IDS was still able to identify a Rank anomaly. This shows that the effectiveness and detectability of a Sinkhole attack depend not only on the advertised Rank, but also on the attacker's position and on the parent-selection mechanisms of the routing protocol.

To visualize the effect of the Sinkhole attack on the routing topology, the DAGs obtained before and after the attack are compared in Fig. 5.

In the scenarios where the DAG changes, the nodes redirected through the Sinkhole generally become one hop farther from the root, since their new routing path passes through the malicious node. This effect is particularly visible in Sinkhole 1, Sinkhole 2, and Sinkhole 4.

In contrast, the DAG of Sinkhole 3 remains unchanged after the attack. Although the Sinkhole is located at a bottleneck position, the existing routing structure and the parent-selection mechanism prevent the advertised Rank from causing a topology change. This result further highlights the influence of the node's position on the effectiveness of the attack.

6.3 IDS effectiveness

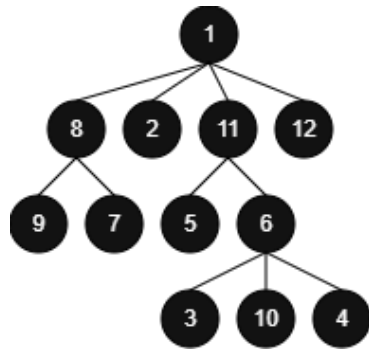
The results show that the effectiveness of the developed IDS depends strongly on the type of anomaly being monitored and on the assumptions made during its design. The proposed approach is intentionally lightweight and rule-based: it extracts relevant events from the Contiki-NG logs, correlates them according to the expected

RPL behavior, and generates an alert when a predefined condition is violated. This follows the general event-processing principles commonly adopted in security monitoring systems, where event collection, correlation, and alerting form the basis for identifying security-relevant activity.

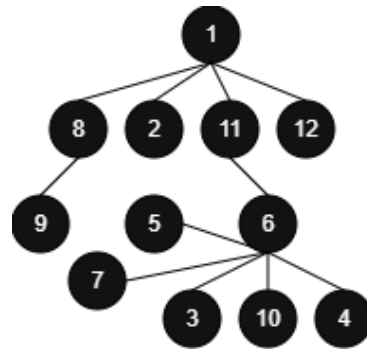
For the Blackhole scenarios, the forwarding-based rule proved effective when the malicious node actually forwarded traffic. However, it could not identify a Blackhole node that had no children and therefore received no packets to forward. This highlights an important limitation of deterministic rules: their effectiveness is directly related to the availability of observable evidence.

A similar limitation emerged for the Sinkhole detection. The Rank-consistency rule successfully identified the manipulated topology when the attack produced an inconsistent parent-child Rank relationship. However, the additional Rank-change rule proved too sensitive, as Rank transitions occurring during normal DODAG construction were also reported as suspicious.

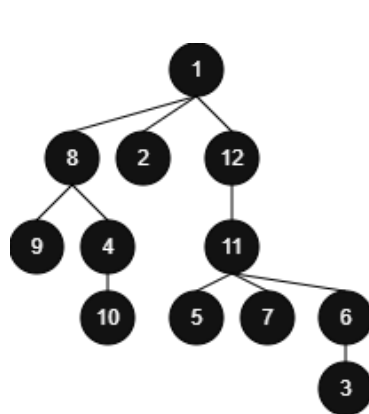
These observations suggest that an IDS should not rely exclusively on isolated events or single threshold-based rules. Correlating multiple observations, considering their temporal context, and distinguishing normal protocol initialization from anomalous behavior can reduce false positives and improve detection reliability. This is consistent with the broader distinction between deterministic rule-based detection and more context-aware correlation approaches discussed in previous security-monitoring work [2].



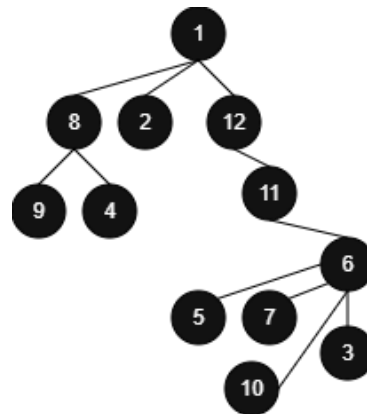
(a) Sinkhole 1 before attack (1 = server, 6 = attacker).



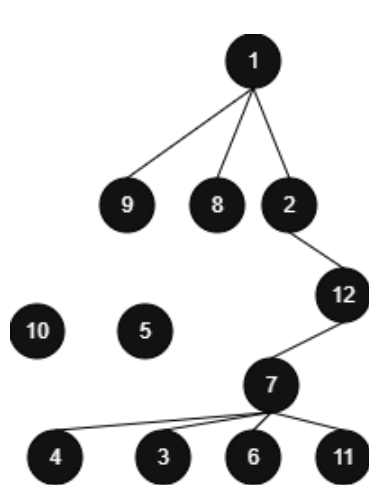
(b) Sinkhole 1 after attack (1 = server, 6 = attacker).



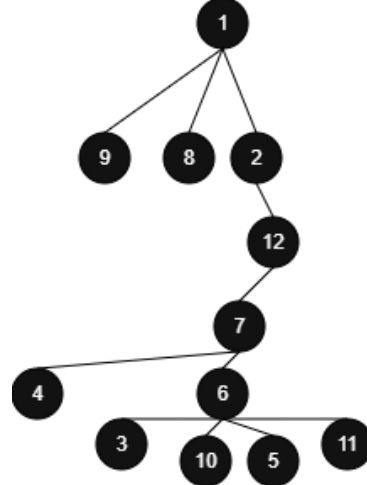
(c) Sinkhole 2 before attack (1 = server, 6 = attacker).



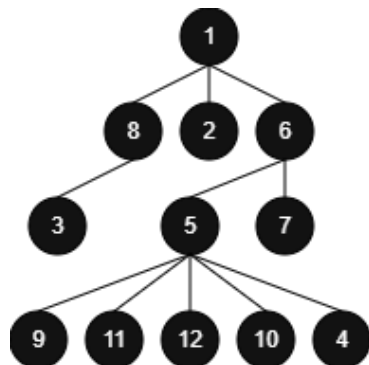
(d) Sinkhole 2 after attack (1 = server, 6 = attacker).



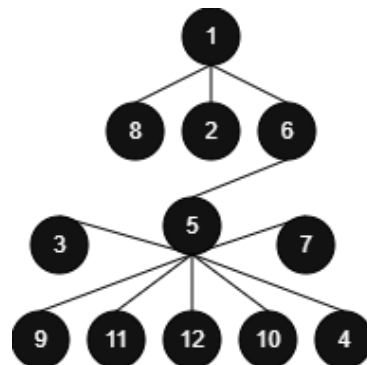
(e) Sinkhole 4 before attack (1 = server, 6 = attacker).



(f) Sinkhole 4 after attack (1 = server, 6 = attacker).



(g) Blackhole 4 before attack (1 = server, 5 = attacker).



(h) Blackhole 4 after attack (1 = server, 5 = attacker).

Figure 5: DAGs before and after the Sinkhole attack.

Acronyms

6LoWPAN IPv6 over Low-Power Wireless Personal Area Networks. 2

DAG Directed Acyclic Graph. 7, 11, 13

DAO Destination Advertisement Object. 3, 7

DAO-ACK Destination Advertisement Object Acknowledgment. 3

DIO DODAG Information Object. 2, 3, 5–7

DIS DODAG Information Solicitation. 2

DODAG Destination-Oriented Directed Acyclic Graph. 2, 3, 5, 7, 8, 12

ETX Expected Transmission Count. 2, 6

ICMPv6 Internet Control Message Protocol for IPv6. 2

IDS Intrusion Detection System. 2–4, 6–9, 11, 12

IETF Internet Engineering Task Force. 2

IoT Internet of Things. 2, 3

IPv6 Internet Protocol Version 6. 2, 4, 5

LLN Low-Power and Lossy Network. 2

MRHOF Minimum Rank with Hysteresis Objective Function. 2, 3, 6, 11

OF Objective Function. 2, 3

OF0 Objective Function Zero. 2

PDR Packet Delivery Ratio. 4, 7–9, 11

RPL Routing Protocol for Low-Power and Lossy Networks. 2–7, 12

UDGM Uniform Distributed Grid Model. 7

UDP User Datagram Protocol. 5, 7

WSL2 Windows Subsystem for Linux 2. 4

References

- [1] Behnam Farzaneh, Mohammad Ali Montazeri, and Shahram Jamali. An anomaly-based IDS for detecting attacks in RPL-based internet of Things. pages 61–66, 2019.
- [2] Emanuele Artusi. Integrazione tassonomia ACN in sistemi SIEM per conformità NIS 2, 2025.